



SEEK - O - SPHERE

Project Report

Seek-O-Sphere

A free, interactive website that teaches Artificial Intelligence to everyone, ages 8 to 80 — built with plain HTML, CSS and JavaScript on a Firebase backend.

Project: Seek-O-Sphere · **Developer:** Soumil

Updated 16 August 2026

1. Introduction

Most explanations of artificial intelligence are written either for children or for engineers. Seek-O-Sphere is an attempt at neither: one site that explains AI in plain language to a ten-year-old and to their grandparent, without talking down to either.

It is free, it needs no installation, and it works on a phone. The whole thing is static HTML, CSS and JavaScript served from GitHub Pages, with Firebase behind it for accounts and progress.

2. Objectives

- Explain AI clearly to learners aged roughly 8 to 80, in one coherent voice.
- Pair reading with practice, so understanding can be tested rather than assumed.
- Track each learner's progress and show it back to them usefully.
- Let a school run its own group without seeing, or being seen by, any other school.
- Cost nothing to host and nothing to use.

3. Scope

The site covers five learning chapters, an interactive history of AI, a practice Lab in two modes, a tool recommender, a searchable help centre, a study planner, personal dashboards, and an administration panel for schools. It deliberately does not attempt: video lessons, live classes, payments, or a mobile app.

4. Technology and Why

Decision	Reasoning
No framework	The site must outlive its dependencies. Plain HTML/CSS/JS has no build step to rot, no <code>node_modules</code> to reinstall, and can be opened and edited by anyone.
GitHub Pages	Free static hosting with versioned deploys. Cost was a design constraint, not an afterthought.
Firebase, compat SDK	Auth and a database with no server to run. The compat SDK specifically: the modular one produced silent failures here, and reliability beat modernity.
Offline persistence	Reads come from cache and writes commit locally first, so a weak connection does not lose a learner's answers.
A local question engine	Algorithmic maths plus a curated JSON bank means practice is free and works offline. An optional Claude API mode exists, with the local bank as automatic fallback.

5. Features Delivered

- **Learning chapters** — What is AI?, History, Types, Study Tools, Using AI Responsibly.
- **Interactive history** — a 1950→2026 scrubber, an era ribbon, a "Then / Now" focus card and a "Guess the Year" quiz.
- **Visual Learning Lab** — Curriculum mode across five boards and five subjects, and Exam Prep for five competitive exams, both with hints, explanations, streaks and saved progress.
- **Find My AI Tool** — a keyword-scored recommender over more than twenty tools.
- **Eulid** — a mascot who can be dragged around any page and whose five legs are working tools: a magnifier, a multi-colour highlighter with an eraser, read-aloud, a dashboard shortcut and a user-defined shortcut.
- **Study planner, student dashboard and admin panel.**

6. Backend Design

Three collections carry the system: `users`, `progress` and `planner`, with `schools` and `joinCodes` supporting the school model. Progress is stored one document per day per mode per subject, updated with atomic increments. That shape keeps reads cheap: a dashboard chart is one narrow query, not a scan of every answer ever given.

7. Security Design

The requirement was strict — a school's administrator must see that school's students and nothing else at all. Firestore cannot filter a list on request; it can only refuse a query whose results it cannot guarantee. The design therefore gives every user document a derived `bucket` field naming exactly one list it belongs to, and school admins query that one value. Because `bucket` is recomputed from `role` and `schoolId` on every write and rejected if it disagrees, a user cannot edit themselves into or out of a school's list.

One limitation is recorded rather than papered over: rules are all-or-nothing per document, and a planner day is one document, so an admin able to add an item to a day is technically able to read it. The application shows admins only the items admins set. That is a decision in code, not a guarantee from the rules, and it is documented as such.

8. Testing and Verification

The project's verification habit is to **measure rather than assert**. Contrast is checked by hiding a single measured line, screenshotting the pixels it covered and reading them back, using the 5th percentile over varying backgrounds rather than a single worst pixel. Page structure is checked comparatively against a pristine baseline by `structure.js`, which exists because a single extra closing tag once closed the home panel early and leaked its content onto every other section — a fault no rendering check of the home page could have caught.

The rule learned from that: **if a check passes on the version you know is broken, suspect the check**. Several checks in this project were tightened rather than loosened after they passed too easily.

9. Results

The site runs at no cost, works offline after first load, and is usable on a phone. Practice is recorded per day and per subject and surfaces in a dashboard the learner can read. A school can be created, joined by code, and administered without any visibility across the school boundary.

10. Future Work

- A greeting animation of Eulid on the home page. The blocker is source material: every export so far has been flattened onto an opaque backdrop, and whichever colour that backdrop is, the parts of the artwork nearest it dissolve when keyed. The fix is an export over a colour the artwork never uses, or one with a real alpha channel.
- Wider curriculum coverage in the question bank.
- Teacher-set assignments in the planner, building on the per-school access already in place.